

Once you have data [loaded](#) and [indexed](#), you will probably want to store it to avoid the time and cost of re-indexing it. By default, your indexed data is stored only in memory.

Persisting to disk

The simplest way to store your indexed data is to use the built-in `.persist()` method of every Index, which writes all the data to disk at the location specified. This works for any type of index.

```
index.storage_context.persist(persist_dir="<persist_dir>")
```

Here is an example of a Composable Graph:

```
graph.root_index.storage_context.persist(persist_dir="<persist_dir>")
```

You can then avoid re-loading and re-indexing your data by loading the persisted index like this:

```
from llama_index.core import StorageContext, load_index_from_storage

# rebuild storage context
storage_context = StorageContext.from_defaults(persist_dir="<persist_dir>")

# load index
index = load_index_from_storage(storage_context)
```

Tip

Important: if you had initialized your index with a custom `transformations`, `embed_model`, etc., you will need to pass in the same options during `load_index_from_storage`, or have it set as the [global settings](#).

Using Vector Stores

As discussed in [indexing](#), one of the most common types of Index is the VectorStoreIndex. The API calls to create the [embeddings](#) in a VectorStoreIndex can be expensive in terms of time and money, so you will want to store them to avoid having to constantly re-index things.

LlamaIndex supports a [huge number of vector stores](#) which vary in architecture, complexity and cost. In this example we'll be using Chroma, an open-source vector store.

First you will need to install chroma:

```
pip install chromadb
```

To use Chroma to store the embeddings from a VectorStoreIndex, you need to:

- initialize the Chroma client
- create a Collection to store your data in Chroma
- assign Chroma as the `vector_store` in a `StorageContext`
- initialize your VectorStoreIndex using that StorageContext

Here's what that looks like, with a sneak peek at actually querying the data:

```
import chromadb
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from llama_index.vector_stores.chroma import ChromaVectorStore
from llama_index.core import StorageContext

# load some documents
documents = SimpleDirectoryReader("./data").load_data()

# initialize client, setting path to save data
db = chromadb.PersistentClient(path="./chroma_db")

# create collection
chroma_collection = db.get_or_create_collection("quickstart")

# assign chroma as the vector_store to the context
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

# create your index
index = VectorStoreIndex.from_documents(
    documents, storage_context=storage_context
)

# create a query engine and query
query_engine = index.as_query_engine()
```

```
response = query_engine.query("What is the meaning of life?")
print(response)
```

If you've already created and stored your embeddings, you'll want to load them directly without loading your documents or creating a new `VectorStoreIndex`:

```
import chromadb
from llama_index.core import VectorStoreIndex
from llama_index.vector_stores.chroma import ChromaVectorStore
from llama_index.core import StorageContext

# initialize client
db = chromadb.PersistentClient(path="./chroma_db")

# get collection
chroma_collection = db.get_or_create_collection("quickstart")

# assign chroma as the vector_store to the context
vector_store = ChromaVectorStore(chroma_collection=chroma_collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

# load your index from stored vectors
index = VectorStoreIndex.from_vector_store(
    vector_store, storage_context=storage_context
)

# create a query engine
query_engine = index.as_query_engine()
response = query_engine.query("What is llama2?")
print(response)
```

Tip

We have a [more thorough example of using Chroma](#) if you want to go deeper on this store.

You're ready to query!

Now you have loaded data, indexed it, and stored that index, you're ready to [query your data](#).

Inserting Documents or Nodes

If you've already created an index, you can add new documents to your index using the `insert` method.

```
from llama_index.core import VectorStoreIndex

index = VectorStoreIndex([])
for doc in documents:
    index.insert(doc)
```

See the [document management how-to](#) for more details on managing documents and an example notebook.